National University of computer and emerging sciences

Database Systems Lab (CL-2005)

Semester: Spring 2026
Section: BCS-4E
Course Instructor: Mr. Sameer Faisal

LAB # 03

Understanding Role of SQL as DDL

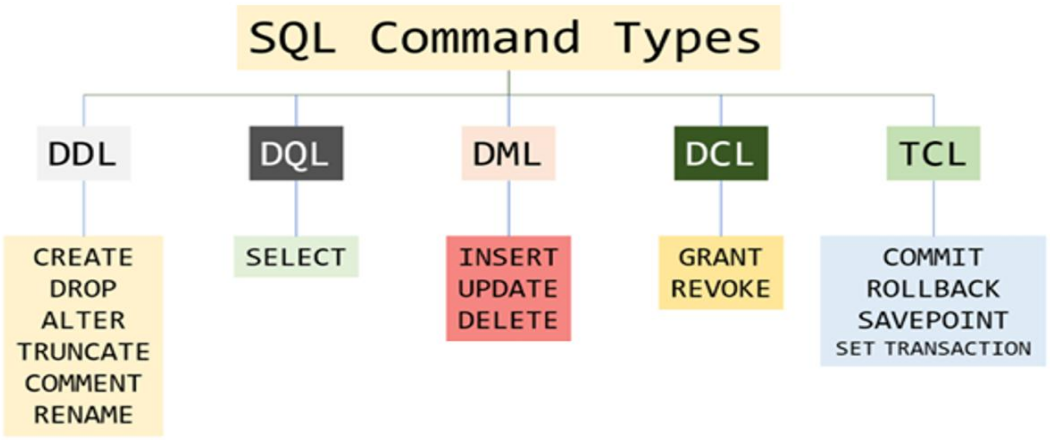
Objectives:

- Explore the role of SQL as DDL in defining and managing database structures.
- Explore SQL data types and understand their appropriate usage.
- Learn the concept of constraints and their importance in enforcing data integrity.
- Learn about PRIMARY KEY and FOREIGN KEY constraints and their importance in table relationships.
- Apply constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and DEFAULT in table creation.
- Learn to insert single and multiple rows into tables using the INSERT INTO statement.
- Build the ability to enforce rules and relationships between database entities effectively.

SQL Command Categories

Structured Query Language (SQL) as we all know is the database language by the use of which we can perform certain operations on the existing database and also, we can use this language to create a database. SQL uses certain commands like Create, Drop, Insert, etc. to carry out the required tasks. These SQL commands are mainly categorized into following categories:

- DDL – Data Definition Language
- DQL – Data Query Language
- DML – Data Manipulation Language
- DCL – Data Control Language
- TCL – Transaction Control Language



SQL as DDL (Data Definition Language)

DDL or Data Definition Language actually consists of the SQL commands that can be used to define the database schema. It deals with the description of the database schema and provides the tools to create, modify, and organize the structure of database objects such as tables, views, and indexes. DDL ensures that the database is properly structured and that the relationships between different entities are maintained correctly. It also allows setting rules for the data using constraints, which help preserve data accuracy and consistency. By using DDL, a database designer can plan how the information will be stored and how different parts of the database will interact with each other.

DDL provides several commands to define and modify the database schema. The most commonly used DDL commands include CREATE, ALTER, DROP, TRUNCATE, COMMENT, and RENAME, each serving a specific purpose in managing database objects. This lab mainly focuses on:

- **CREATE** – Used to create database and its objects such as tables, indexes, views, stored procedures, functions, and triggers.
- **ALTER** – Used to modify the structure of an existing database or its objects, such as tables, columns, or constraints.
- **COMMENT –** Used to add descriptive notes to database objects in the data dictionary, helping users and developers understand the purpose of tables and columns without affecting the data or structure.
- **RENAME –** Used to change the name of an existing database object in the schema without affecting its data or structure.

In this lab manual, we will go through each DDL command step by step, understand what it does, and learn how to use it to design and organize database structures efficiently.

Data Types in SQL

After understanding the main DDL commands, the next step is to learn about the different data types available in SQL. Data types define the kind of information that can be stored in a column and help ensure that the data is accurate, consistent, and efficiently stored.

Choosing the correct data type is important for proper database design, as it affects storage space, performance, and the way data is processed.

In this section, we will explore the commonly used SQL data types, their purposes, and examples of how they can be applied when creating tables.

Below listed are some commonly used data types in SQL and their purpose:

Data Type	Description	Example
INT	Stores whole numbers.	Employee age: 25
BIGINT	Stores very large whole numbers.	Population: 9876543210
DECIMAL / NUMERIC	Stores fixed-point numbers with exact precision.	Product Price: 345.25
REAL / FLOAT	Stores approximate floating-point numbers.	Value of Pi(π): 3.14159
MONEY / SMALLMONEY	Stores currency values.	Salary: 7500.75
DATE	Stores only date values.	Birthdate: 20-SEP-1999
TIME / DATETIME	Stores time or both date and time.	Meeting: 2026-01-25 14:30
CHAR(n)	Stores fixed-length text.	Gender: 'M' (CHAR(1))
NCHAR(n)	Stores fixed-length Unicode text.	Grade symbol: N'Ü'
VARCHAR(n)	Stores variable-length text.	Name: 'Hammad'
NVARCHAR(n)	Stores variable-length Unicode text.	Name in Urdu: N'علی'
BINARY / VARBINARY(n)	Stores fixed or variable-length binary data.	File checksum: 0xFA01
BIT	Stores boolean values (0 or 1).	IsActive: 1 (True)

Using the CREATE Command in SQL

CREATE command is used to create databases and database objects such as tables, indexes, views, stored procedures, functions, and triggers. For beginners, the most important use of CREATE is to define a table, where you specify:

- Column Names (Attributes/Fields).
- Data Types (what kind of data can be stored).
- Constraints (if any).

By using CREATE, a database designer can structure information properly and ensure that data is stored in an organized way.

There are two CREATE statements mainly available in SQL:

1. CREATE DATABASE

Example Syntax:

```
CREATE DATABASE database_name;
```

2. CREATE TABLE

Example Syntax:

```
CREATE TABLE table_name(
column1 datatype(size),
column2 datatype(size),
column3 datatype(size)
);
```

Example Usage:

```
CREATE TABLE Students (
Student_ID  INT,
Student_Name  VARCHAR(50),
Student_Age  INT,
Enrollment_Date DATE,
Student_Email NVARCHAR(150)
);
```

Student Task – 01: Create a table of your choice containing at least 7-8 attributes (columns) selecting the relevant table name and attribute details along with proper usage of datatypes for each attribute. **(Submit your work on Google Classroom at the end of the lab)**

Enforcing Data Rules with Constraints

In SQL, a constraint is a rule or restriction applied to a column or table to ensure that the data stored in the database is accurate, consistent, and reliable. Constraints help the database maintain data integrity by enforcing rules on what values can be entered, how records are related, and how duplicates are prevented.

Key Points:

- **Constraints prevent errors** – e.g., ensuring no two rows have the same unique ID.
- **Constraints control what data is allowed** – e.g., a column cannot be empty if NOT NULL is applied.
- **Constraints maintain relationships** – e.g., PRIMARY KEY and FOREIGN KEY ensures data in one table matches or relates or connects to another table.

Constraints can be applied when a table is created using the CREATE TABLE statement, or they can be added later to an existing table using the ALTER TABLE statement.

Applying constraints ensures that the data in the table remains accurate, consistent, and reliable.

The following constraints are commonly used in SQL:

- NOTNULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK
- DEFAULT

Constraint Usage in SQL

1. NOT NULL

Ensures that a column cannot have a NULL value. This means a value must be provided for the column when inserting a record. It is commonly used for important fields where missing data is not allowed.

Example Usage

```
CREATE TABLE Table_Name(
Column1 datatype NOT NULL,
Column2 datatype NOT NULL,
Column3 datatype
);
```

If a value is not provided for a NOT NULL column during insertion, the database will reject the operation and generate an error.

2. UNIQUE

Ensures that all values in a column are different. This means duplicate values are not allowed in the column. However, a UNIQUE column can contain a NULL value unless NOT NULL is also specified.

Example Usage

```
CREATE TABLE Table_Name(
Column1 datatype UNIQUE NOT NULL,
Column2 datatype UNIQUE,
Column3 datatype NOT NULL
);
```

UNIQUE is commonly used for fields like email addresses or phone numbers.

3. DEFAULT

Sets a default value for a column when no value is provided during record insertion. If the user does not specify a value for the column, the database automatically assigns the default value. If a value is provided, the default value is ignored. It is commonly used for columns like status, date, or flags.

Example Usage

```
CREATE TABLE Students (
StudentID INT,
StudentName VARCHAR(50),
Status VARCHAR(20) DEFAULT 'Active',
AdmissionDate DATE DEFAULT GETDATE()
);
```

DEFAULT helps reduce missing data and simplifies data insertion.

4. CHECK

Ensures that the values in a column satisfy a specific condition. This constraint allows only those values that meet the defined rule, helping maintain data validity.

Example Syntax

```
CREATE TABLE Table_Name(
Column1 datatype UNIQUE NOT NULL,
Column2 datatype UNIQUE,
Column3 datatype NOT NULL
CHECK (Column Condition)
);
```

Example Usage

```
CREATE TABLE Students (
   StudentID INT,
   StudentName VARCHAR(50),
   Age INT CHECK (Age >= 18),
   Marks INT CHECK (Marks BETWEEN 0 AND 100)
);
```

CHECK is commonly used to restrict values within a range, such as age greater than 0 or marks between 0 and 100.

Another Example:

```
CREATE TABLE Students (
   StudentID INT,
   StudentName VARCHAR(50),
   Gender VARCHAR(10)
   CHECK (Gender IN ('Male', 'Female'))
);
```

CHECK with IN keyword is useful when a column should accept only predefined values.

5. PRIMARY KEY

A PRIMARY KEY is a constraint that is used to uniquely identify each record (row) in a table. It ensures that every row in the table can be distinguished from all others.

A table can have only one PRIMARY KEY, but that key can consist of one or more columns.

A Column Defined as PRIMARY KEY:

- Cannot have **duplicate values.**
- Cannot contain **NULL values.**
- Must **uniquely** identify each record in the table.

Why PRIMARY KEY is Important:

- Uniquely identifies each row.
- Prevents duplicate records.
- Helps create relationships with other tables.
- Improves data organization and integrity.

Example Usage

```
CREATE TABLE Students (
   StudentID INT PRIMARY KEY,
   StudentName VARCHAR(50),
   Age INT,
   Email VARCHAR(100)
);
```

- **StudentID uniquely identifies each student.**
- **No two students can have the same StudentID.**
- **StudentID cannot be NULL.**

Another Example

```
CREATE TABLE Students (
   StudentID INT,
   StudentName VARCHAR(50),
   Age INT,
   Email VARCHAR(100),
   PRIMARY KEY (StudentID)
);
```

6. FORIEGN KEY

A FOREIGN KEY is a constraint that is used to create a link between two tables. It ensures that the value in one table must exist in another related table.

In simple words, a FOREIGN KEY refers to the PRIMARY KEY of another table and maintains relationship and consistency between tables.

For example, A student belongs to a department, so the department must exist before assigning it to a student.

Why FOREIGN KEY is Important:

- Maintains data consistency.
- Prevents invalid data entry.
- Enforces relationships between tables.
- Avoids orphan records.

Example Usage

```
CREATE TABLE Departments (
   DepartmentID INT PRIMARY KEY,
   DepartmentName VARCHAR(50)
);

CREATE TABLE Students (
   StudentID INT PRIMARY KEY,
   StudentName VARCHAR(50),
   DepartmentID INT,
   FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

- **DepartmentID in Students is a FOREIGN KEY.**
- **It refers to DepartmentID in Departments table.**
- **A student cannot be assigned to a department that does not exist.**

PRIMARY KEY identifies a record, while FOREIGN KEY connects records between tables.
PRIMARY KEY exists in the parent table, and FOREIGN KEY exists in the child table.

Using the ALTER Command in SQL

The ALTER command in SQL is a DDL (Data Definition Language) command used to modify the structure of an existing table.

With the ALTER command, you can:

- Add new columns to a table.
- Modify existing columns (e.g., change data type or size).
- Remove columns from a table.
- Add or remove constraints (e.g., PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL).

The ALTER command is a tool to change the table structure without deleting the table.

Example Usage – 1:

```
ALTER Table1 MODIFY Column_Name datatype NULL;
```

It's mainly for changing a column's data type or NULL/NOT NULL property.

Example Usage – 2:

```
ALTER TABLE table1 ADD CONSTRAINT const_N UNIQUE (Column1, Column2);
```

This is an example of adding a UNIQUE constraint to an existing table using the ALTER TABLE command.

Example Usage – 3:

ALTER TABLE table1 ADD PRIMARY KEY (Column1);

This example is used to add a PRIMARY KEY constraint to an existing table using ALTER TABLE. It makes Column1 the unique identifier for all rows in the table.

Example Usage – 4:

ALTER TABLE table1 ADD FOREIGN KEY (Column_Name) REFERENCES table2 (Column_Name);

This command adds a FOREIGN KEY to table1, linking its column to the primary key column of table2 to maintain referential integrity.

Example Usage – 5:

ALTER TABLE table1 ADD CHECK (Column_Condition);

OR

ALTER TABLE table1 ADD CONSTRAINT Constraint_Name CHECK (Condition);

These commands add a CHECK constraint to a table to enforce a condition on column values and maintain data integrity.

Using the RENAME Command in SQL

The RENAME command in SQL is used to change the name of an existing table or column without altering its data. Sometimes we may want to rename our table to give it a more relevant name. For this purpose, we can use ALTER TABLE to rename the name of table.

Example Usage:

ALTER TABLE old_table_name RENAME TO new_table_name;

To rename a column, we can also use the ALTER command.

Example Usage:

ALTER TABLE table_name RENAME COLUMN old_name TO new_name;

Inserting Data in Tables Using SQL

The INSERT INTO statement in SQL is used to add new rows to a table. It can be used in two ways: by specifying the column names explicitly, or by inserting values for all columns in the table.

Values Only:

INSERT INTO table_name VALUES (value1, value2, value3….);

table_name: name of the table

value1, value2, value3….: value for first column, second column, third column….for the new record.

Values with Column Name Specified:

INSERT INTO table_name (column1, column2, column3….) VALUES (value1, value2, value3….);

column1, column2, column3….: name of first column, second column, third column …

value1, value2, value3 : value of first column, second column,… for the new record.

You can also insert multiple rows at once using a single INSERT INTO command.

Example Usage:

```
INSERT INTO Students (StudentID, Name, Age)
VALUES
    (1, 'Ali', 18),
    (2, 'Sara', 19),
    (3, 'Ahmed', 20);
```

THE END!